

AMAZON VPC TROUBLESHOOTING PROJECT: EC2 PUBLIC ACCESS AND CONNECTIVITY

BY:

**MOSHOD MARIAM OMOSHALEWA
(RILEWA)**

DATE: 21ST JANUARY, 2026

Introduction: Why This Project Exists

When people talk about Amazon VPC problems, it is usually around the basics. How to create a VPC, how to add subnets, how to launch an EC2 instance. Those things are important, but in real life, they are not the actual problem.

The real Amazon VPC problems show up after everything has already been created:

- Connectivity not working.
- Instances that cannot access the internet.
- Resources that are either too exposed or completely locked down.
- Traffic being blocked and no one understanding why.
- Security mistakes caused by simple misunderstandings of how VPC components work together.

Most people can create a VPC, but many struggle to properly design it, secure it, and troubleshoot it when things go wrong. That is where real-world cloud issues happen.

This project exists to focus on that reality.

VPC troubleshooting is a critical skill for anyone going into cloud or cloud security. It is not just about building infrastructure, but understanding how traffic flows, why access fails, and how small misconfigurations can completely break functionality or introduce security risks.

In this project, I intentionally created multiple VPC misconfigurations to simulate what happens in real environments. The screenshots and issues shown are not accidental. They were deliberately introduced to mirror common mistakes seen in production environments.

No real organization is involved.

The domain name company.com is fictional.

All configurations were created in a controlled environment for learning purposes.

Project Scenario

An organization named Company.com deployed an EC2 instance inside an Amazon VPC for internal testing. The instance is unable to access the internet and cannot be accessed remotely. Further investigation reveals multiple VPC configuration issues affecting routing, security, and connectivity.

The goal of this project is to investigate these issues, identify the root causes, and apply the correct fixes. Both the misconfigurations and the remediation steps were performed by me as part of this simulation, to reflect real-life cloud troubleshooting scenarios.

Project Scope and Objectives

For this project, I simulated common Amazon VPC configuration failures that occur in real-world cloud environments. The focus is on understanding why EC2 connectivity fails and how those failures can be properly identified and fixed.

The objective of this project is to investigate EC2 instances that cannot access the internet or be accessed remotely, identify the underlying VPC misconfigurations, and apply the correct fixes using proper subnet design, route table associations, internet gateway configuration, and security group rules.

Rather than simply creating a functional VPC, this project intentionally introduces misconfigurations first, then documents the troubleshooting and remediation process. This approach reflects how cloud connectivity issues are typically encountered and resolved in real environments.

Environment Overview

This project was carried out using an AWS Free Tier account.

The following AWS services and components were used:

- Amazon VPC
- Subnets
- Route tables
- Internet Gateway
- EC2 instance
- Security groups

At the time of setup, the environment was not designed to be perfect. Instead, misconfigurations were intentionally introduced to simulate realistic troubleshooting scenarios. Subnet placement and routing behavior are analyzed during the investigation rather than assumed upfront.

No architectural diagram was created initially. The environment is documented using screenshots and explanations of how each component was configured and how it impacted connectivity.

Initial Environment Setup and Misconfigurations

To begin the project, I created an Amazon VPC using IPv4 only. The VPC was named “Broken-vpc” and assigned a CIDR block of 10.0.0.0/16. This VPC serves as the base environment where all misconfigurations were intentionally introduced.

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

Broken-vpc

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input ☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block ☐ IPAM-allocated IPv6 CIDR block ☐ Amazon-provided IPv6 CIDR block ☐ IPv6 CIDR owned by me

Initial VPC setup (Broken VPC with CIDR 10.0.0.0/16)

Next, I created only one subnet inside this VPC, named “Broken-subnet”. No specific availability zone preference was selected during creation. This design choice was intentional: relying on a single subnet is not considered best practice and can lead to both availability and connectivity issues.

Additionally, the subnet was not configured as a public subnet, and auto-assign public IPv4 addresses was disabled. At this stage, no steps were taken to allow public internet access.

☰ [VPC](#) > [Subnets](#) > Create subnet

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

Broken-subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference ▼

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16 ▼

IPv4 subnet CIDR block

10.0.1.0/24 256 IPs

< > ^ v

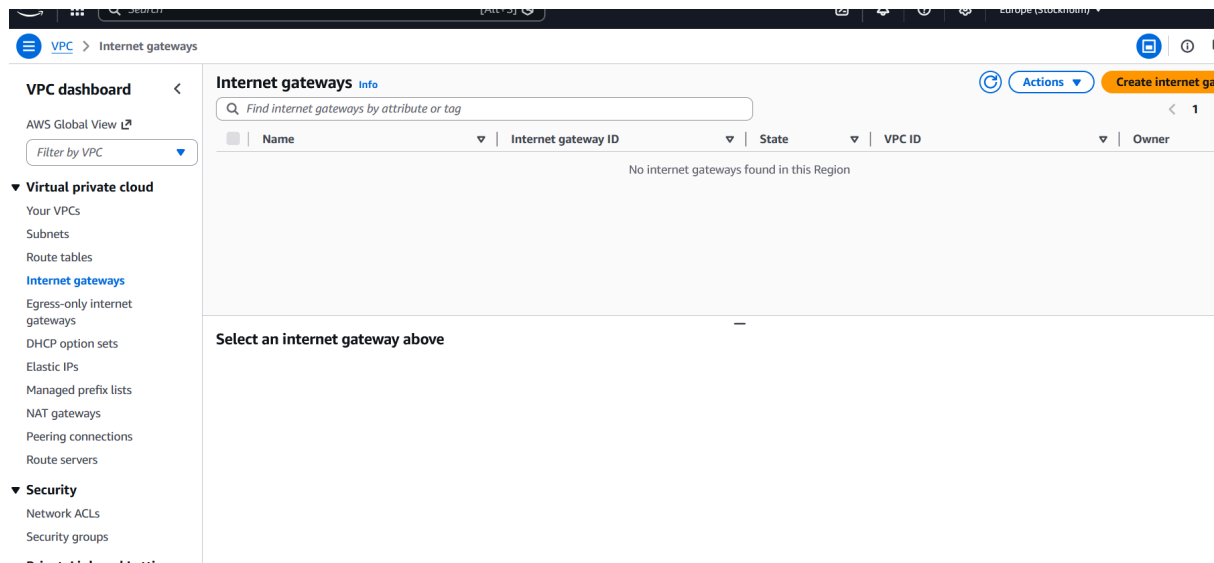
▼ **Tags - optional**

Key	Value - optional
Q Name	Q Broken-subnet

✕ ✕ (Remove)

Created single subnet (Broken Subnet)

When reviewing the internet gateway, it was clear that no internet gateway existed for the VPC. Without an internet gateway, the VPC has no path to the internet, immediately creating a connectivity failure for any resources inside it.



No Internet Gateway attached

Reviewing the route table associated with the subnet revealed further misconfiguration. The subnet used the default route table, which contained only a single route: the destination 10.0.0.0/16 with the target set to local. There was no 0.0.0.0/0 route directing traffic to an internet gateway, confirming another misconfiguration that prevents outbound internet access.

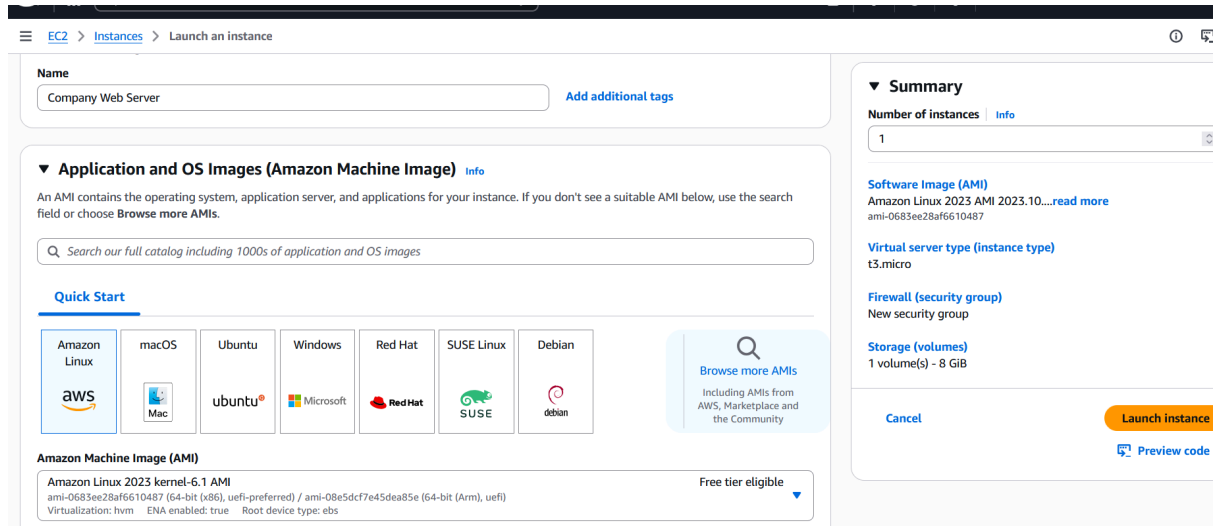
The screenshot displays the AWS VPC console interface. On the left, the 'VPC dashboard' sidebar lists various VPC resources. The main area is titled 'Route tables (1/2)' and shows a table of route tables. The first route table, 'rtb-0430856bd6dc1bd7a', is selected. Below the table, the 'Routes' tab for this route table is active, showing a single route with destination '10.0.0.0/16' and target 'local'.

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
-	rtb-0430856bd6dc1bd7a	-	-	Yes	vpc-0c08526cc3aefad91 Broke.
-	rtb-097047ab34a9ff555	-	-	Yes	vpc-09bfd8f4b8a4f2ab8

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Default route table configuration
(only local route)

After setting up these networking components, I launched an EC2 instance inside Broken-subnet. The instance used Amazon Linux with a T3 Micro instance type. A key pair was created for SSH access, and a security group was attached. Initially, the security group appeared reasonable because it allowed inbound SSH access from my IP address.



EC2 instance launch inside Broken Subnet

However, the security group's outbound rules were misconfigured: all outbound traffic was removed, leaving the instance with no outbound access at all. This means the instance could not reach the internet or any external service, regardless of routing or gateway configuration.

IP addresses available: 251 CIDR: 10.0.1.0/24

Auto-assign public IP [Info](#)

Disable

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

Security group name - *required*

web-ec2-sg

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-./()#,@!+=&;[]\$*

Description - *required* [Info](#)

launch-wizard-1 created 2026-01-18T21:50:51.837Z

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 102.89.75.123/32) [Remove](#)

Type Info	Protocol Info	Port range Info
ssh	TCP	22

Source type Info	Name Info	Description - <i>optional</i> Info
My IP	Add CIDR, prefix list or security group	e.g. SSH for admin desktop

102.89.75.123/32

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.10...[read more](#)

ami-0685ee28af6610487

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel [Launch instance](#) [Preview code](#)

Security group with restricted
outbound rules

EC2 > Security Groups > sg-0d895fce25ede29a8 - web-ec2-sg > Edit outbound rules

Edit outbound rules [Info](#)

Outbound rules control the outgoing traffic that's allowed to leave the instance.

Outbound rules [Info](#)

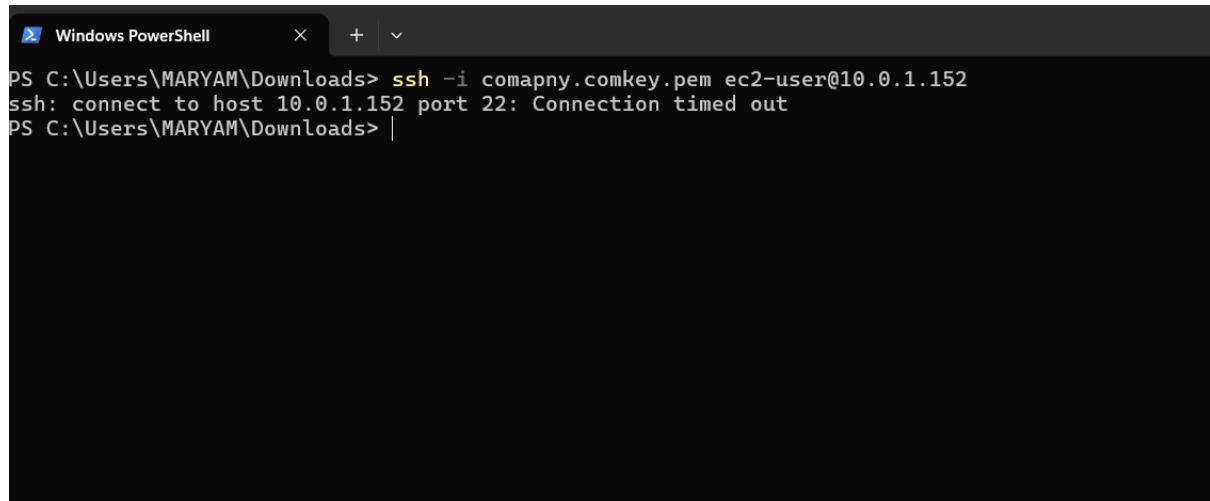
This security group has no outbound rules.

[Add rule](#)

Cancel [Preview changes](#) [Save rules](#)

Outbound traffic completely
blocked

To confirm the failure, I attempted to SSH into the EC2 instance using the configured key pair. The connection attempt resulted in a timeout error, indicating the instance was unreachable.

A screenshot of a Windows PowerShell terminal window. The title bar shows 'Windows PowerShell' with standard window controls. The command prompt shows the user is in the directory 'C:\Users\MARYAM\Downloads'. The command entered is 'ssh -i comapny.comkey.pem ec2-user@10.0.1.152'. The output shows 'ssh: connect to host 10.0.1.152 port 22: Connection timed out'. The prompt returns to 'PS C:\Users\MARYAM\Downloads>'.

```
PS C:\Users\MARYAM\Downloads> ssh -i comapny.comkey.pem ec2-user@10.0.1.152
ssh: connect to host 10.0.1.152 port 22: Connection timed out
PS C:\Users\MARYAM\Downloads> |
```

Failed SSH attempt (Connection
timeout)

Finally, a review of the EC2 instance details confirmed that no public IPv4 or IPv6 address had been assigned. Since the instance was not in a public subnet and had no public IP, there was no possible path for external access or internet connectivity.

The screenshot shows the AWS Management Console interface for connecting to an EC2 instance. The breadcrumb navigation at the top reads: EC2 > Instances > i-0945ef9350d87a6c1 > Connect to instance. The main heading is 'Connect' with an 'Info' icon. Below it, a message says 'Connect to an instance using the browser-based client.' There are four tabs: 'EC2 Instance Connect' (selected), 'Session Manager', 'SSH client', and 'EC2 serial console'. Two yellow warning boxes are present: the first states 'No public IPv4 or IPv6 address assigned' and the second states 'Instance is not in public subnet'. Below these, the 'Instance ID' is listed as 'i-0945ef9350d87a6c1 (Company Web Server)'. Under 'Connection type', there are two radio buttons: 'Connect using a Public IP' (selected) and 'Connect using a Private IP'. Under the 'Public IP' option, there are radio buttons for 'Public IPv4 address' and 'IPv6 address', both of which are disabled. A 'Username' field is at the bottom.

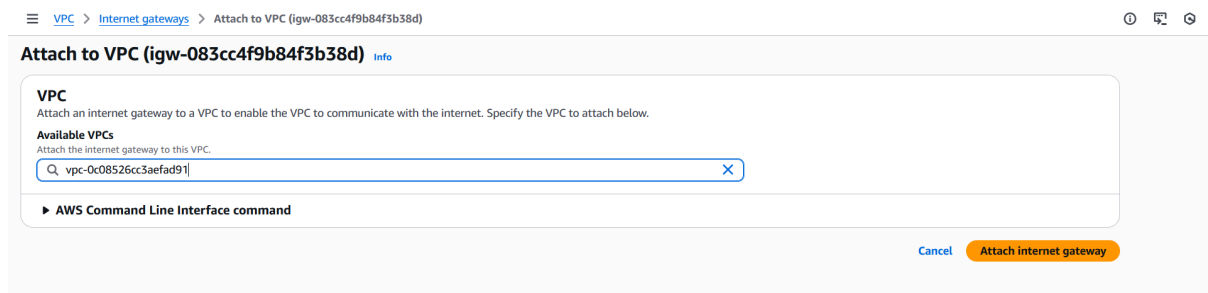
Instance has no public IP
(connectivity failure)

At this stage, the EC2 instance was completely unreachable and had no internet access. The next step is to analyze these failures, identify the root causes, and apply the appropriate fixes.

Fixes for VPC Connectivity Issues

Internet Gateway Missing

The VPC had no internet gateway attached, which meant there was no path to the internet and no north-south traffic possible. To fix this, we created an internet gateway and attached it to the VPC.

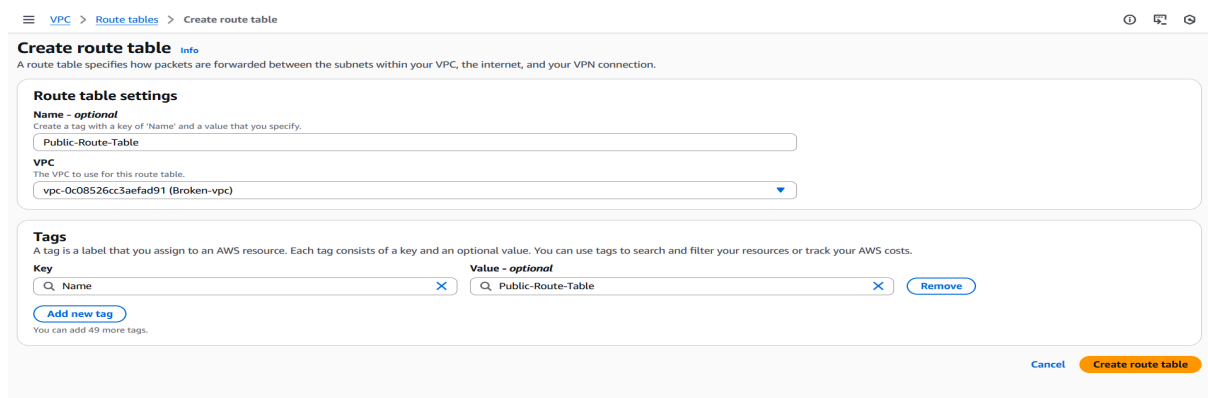


The screenshot shows the AWS Management Console page titled "Attach to VPC (igw-083cc4f9b84f3b38d)". The breadcrumb navigation is "VPC > Internet gateways > Attach to VPC (igw-083cc4f9b84f3b38d)". The page has a section for "VPC" with instructions to attach an internet gateway. Below this is a section for "Available VPCs" with a search bar containing "vpc-0c08526cc3aefad91". At the bottom right, there are "Cancel" and "Attach internet gateway" buttons.



Route Table Misconfiguration

A route table is essential for directing traffic between subnets, the internet, and any VPN connections. We created a new route table and edited it to include a route with destination 0.0.0.0/0 and set the target to the internet gateway. This allows traffic from the subnet to reach the internet.



The screenshot shows the AWS Management Console page titled "Create route table". The breadcrumb navigation is "VPC > Route tables > Create route table". The page has a section for "Route table settings" with a "Name" field containing "Public-Route-Table" and a "VPC" dropdown menu showing "vpc-0c08526cc3aefad91 (Broken-vpc)". Below this is a "Tags" section with a table for key-value pairs. The table has one row with key "Name" and value "Public-Route-Table". At the bottom right, there are "Cancel" and "Create route table" buttons.



☰ [VPC](#) > [Route tables](#) > [rtb-Qe2bf37f97c8dd0b9](#) > Edit routes

🔍

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	local	-	No	CreateRoute
	Internet Gateway			
	igw-083cc4f9b84f3b38d			

Edited route table to include
0.0.0.0/0 to IGW

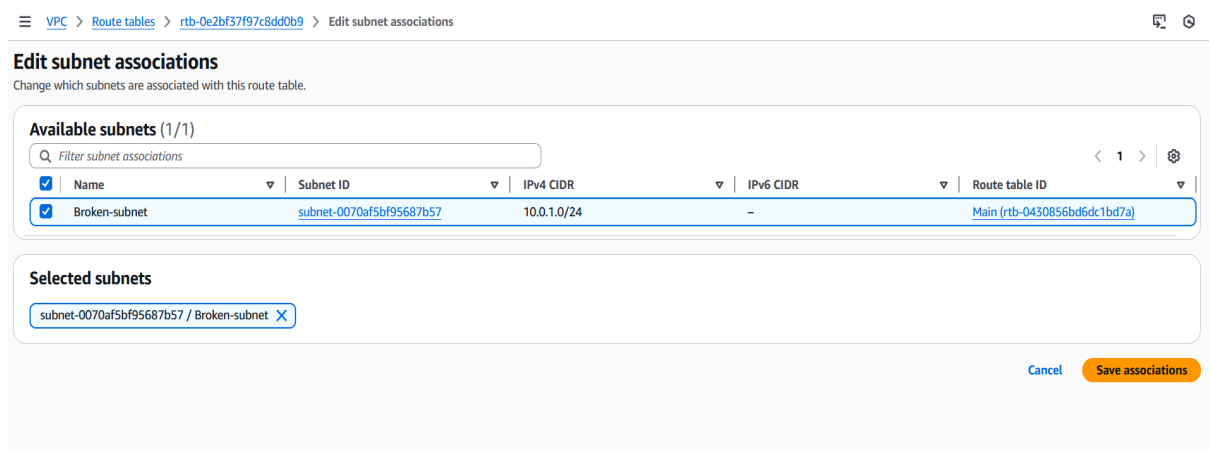
Subnet Association

A route table does nothing on its own. Creating a route table and adding routes is not enough. The route table must be associated with a subnet before it can affect traffic.

Subnets decide which route table they follow. If a subnet is not associated with the correct route table, it will continue using the default one, even if a new route table exists.

That is why, after creating the public route table and adding the 0.0.0.0/0 route to the Internet Gateway, the subnet had to be explicitly associated with that route table. Without this association, outbound traffic would still have no valid path to the internet.

That association step is what actually makes the subnet behave like a public subnet.



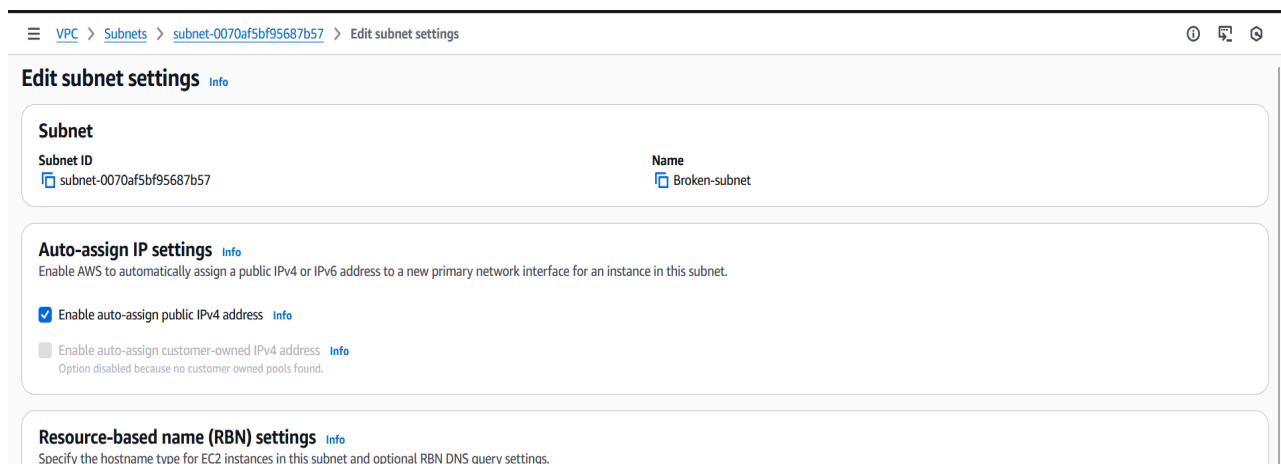
Associated subnet with the public
route table

Public IP Assignment

Even with an Internet Gateway and a correct route table, an EC2 instance still needs a public IPv4 address to communicate with the internet.

Because this VPC was IPv4-only, the subnet had to be configured to automatically assign public IPv4 addresses to instances launched inside it. Without a public IP, the instance has no way to send or receive traffic from the internet, even if routing is correct.

By enabling auto-assign public IPv4 on the subnet, AWS automatically assigns a public IP address to the EC2 instance, allowing it to communicate with the internet through the Internet Gateway.



Enabled auto-assign Public IPv4 on
subnet

Security Group Outbound Rule

Routing alone is useless if outbound traffic is blocked. The security group attached to the instance had all outbound rules removed, preventing any traffic from leaving the instance. We added an outbound rule to allow traffic to flow, ensuring the EC2 instance could reach the internet.

EC2 > Security Groups > sg-Od895fce25ede29a8 - web-ec2-sg > Edit outbound rules

Edit outbound rules

Outbound rules control the outgoing traffic that's allowed to leave the instance.

Security group rule ID	Type	Protocol	Port range	Destination	Description - optional	
-	All traffic	All	All	Anywh... 0.0.0.0/0		Delete

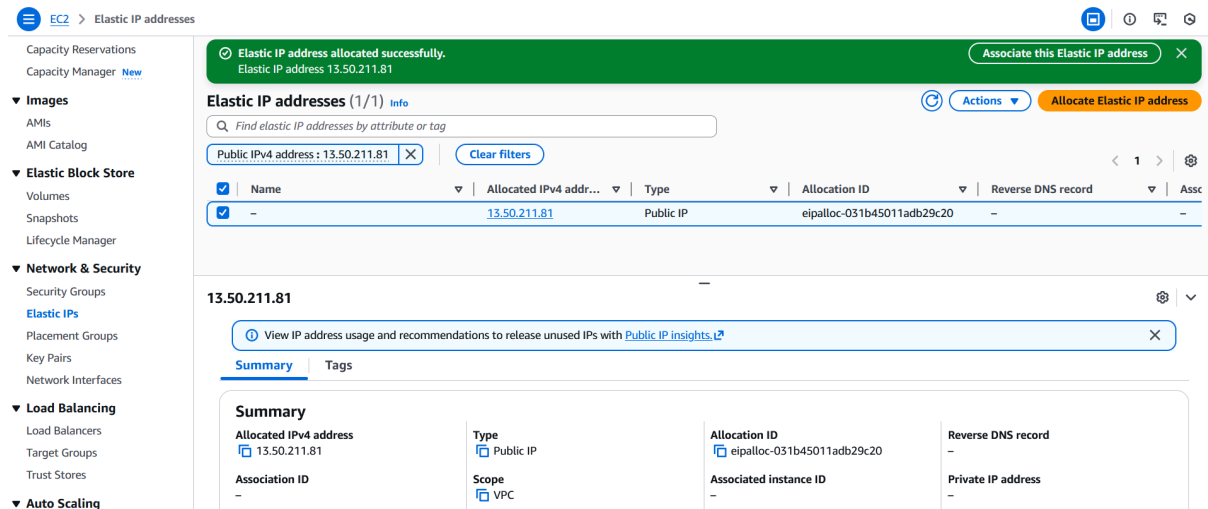
[Add rule](#)

[Cancel](#) [Preview changes](#) [Save rules](#)

Created outbound rule in security group

Elastic IP and Validation of Connectivity

Alright, so after setting up the subnet, route table, and enabling auto-assign public IPv4, we ran into a little hiccup. The public IP still wasn't showing on the network tab for our EC2 instance. Yeah, I know, frustrating. So what did we do? We created an Elastic IP address.



Allocated Elastic IP

For those who aren't familiar, an Elastic IP is basically a static public IPv4 address that you can associate with your EC2 instance. Unlike the auto-assigned public IP, which can change if the instance is stopped or restarted, an Elastic IP stays put. This makes it super useful for servers that need a consistent public address.

So, we created the Elastic IP and then we associated it with the EC2 instance, which I had creatively named Company Web Server. Once we did that, voila, the instance finally showed a public IPv4 address. At this point, everything that needed to be set up for internet access was in place.

EC2 > Elastic IP addresses > Associate Elastic IP address

Associate Elastic IP address [Info](#)

Choose the instance or network interface to associate to this Elastic IP address (13.50.211.81)

Elastic IP address: 13.50.211.81

Resource type
Choose the type of resource with which to associate the Elastic IP address.

☒ Instance
☐ Network interface

Warning: If you associate an Elastic IP address with an instance that already has an Elastic IP address associated, the previously associated Elastic IP address will be disassociated, but the address will still be allocated to your account. [Learn more](#)

If no private IP address is specified, the Elastic IP address will be associated with the primary private IP address.

Instance

Use: "i-0945ef9350d87a6c1"

i-0945ef9350d87a6c1 (Company Web Server) - running

Reassociation
Specify whether the Elastic IP address can be reassociated with a different resource if it already associated with a resource.

☐ Allow this Elastic IP address to be reassociated

[Cancel](#) [Associate](#)

Associated Elastic IP with EC2
instance

Now it was time for validation. We went back to PowerShell and tried SSH again. And guess what? It worked. We got the key pair authentication, the SSH 256 fingerprint, and logged in as the EC2 user without a single timeout. But we didn't stop there, we also pinged **google.com**. And yes, it worked. Internet access was flowing, SSH was working, and the instance finally had a public IP. Everything was finally behaving exactly as it should.

```
PS C:\Users\MARYAM\Downloads> ssh -i comapny.comkey.pem ec2-user@10.0.1.152
ssh: connect to host 10.0.1.152 port 22: Connection timed out
PS C:\Users\MARYAM\Downloads> ssh -i comapny.comkey.pem ec2-user@10.0.1.152
ssh: connect to host 10.0.1.152 port 22: Connection timed out
PS C:\Users\MARYAM\Downloads> ssh -i comapny.comkey.pem ec2-user@13.50.211.81
The authenticity of host '13.50.211.81 (13.50.211.81)' can't be established.
ED25519 key fingerprint is SHA256:2zjxdXlpAL...
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.50.211.81' (ED25519) to the list of known hosts
```

```
      #_
    _#_   #####_   Amazon Linux 2023
   ~\_\_#####\_
  ~~\_\_#####\_
  ~~\_\_####|
  ~~\_\_#\/_---
     ~~~V~' '→
         ~~~~~
          ~.._
           _/_/_
            _/m/'
```

```
[ec2-user@ip-10-0-1-152 ~]$ ping google.com
PING google.com (172.217.19.238) 56(84) bytes of data:
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=1 ttl=118
64 bytes from par21s11-in-f14.1e100.net (172.217.19.238): icmp_seq=2 ttl=118
64 bytes from par21s11-in-f14.1e100.net (172.217.19.238): icmp_seq=3 ttl=118
```

Successful SSH and Internet connectivity test

Conclusion

Honestly, this project was an eye-opener. I mean, I thought I knew Amazon VPCs from studying for the Cloud Practitioner exam, but let me tell you, some things had to be relearned, like the Elastic IP part. That really stood out, showing how important it is to have a static public IP when your instance needs a predictable address.

So yeah, that's it. SSH works, internet works, the instance has a public IP, and the VPC is finally behaving like it should. Learned a ton, had some fun troubleshooting, and honestly, this project was gold.

Reminder: Always remember to delete your AWS resources once you're done with a project, unless you plan to keep them for future use. Leaving resources running can cost money and clutter your account, so don't skip this step.